Identifying Key Entities in Recipe Data

Named Entity Recognition on Recipe Ingredient Lists using Conditional Random Fields (CRF)

Prepared by: M. Bulah

Assignment: Identifying Key Entities in Recipe Data

Course: UpGrad C5 — Identifying Key Entities in Recipe Data

Table of Contents

1. Introduction & Problem Statement

Recipe websites and cooking apps typically store ingredient lists as unstructured free text, for example “2 cups rice cooked 3 tomatoes teaspoons chana dal white urad”. While easy for a human cook to read, this raw text is not directly usable by downstream systems such as dietary trackers, grocery-list generators, or e-commerce ingredient search, all of which need to know precisely which words represent a quantity (“2”), which represent a unit of measure (“cups”, “teaspoons”), and which represent the ingredient itself (“tomatoes”, “chana dal”).

The objective of this assignment is to build a Named Entity Recognition (NER) system that automatically labels every word (token) in a raw recipe ingredient string as one of three entity types — quantity, unit, or ingredient — using a Conditional Random Field (CRF), a sequence-labelling model that is well suited to problems where the label of a token depends heavily on its neighbours.

A correctly trained model of this kind can power a structured recipe/ingredient database that downstream applications (recipe management systems, dietary tracking apps, e-commerce grocery platforms) can query directly, instead of having to parse free text themselves.

2. Data Description & Assumptions

The dataset (ingredient_and_quantity.json) contains 285 recipe records. Each record has two fields:

Field	Description
input	Raw, whitespace-tokenised ingredient list text extracted from a recipe.
pos	A space-separated sequence of NER labels (one per input token) indicating whether that token is a quantity, unit, or ingredient.

2.1 Dataset Summary (Post-Cleaning)

The table below summarises how the raw dataset was reduced during cleaning and how the resulting clean set was split for training and validation (see Section 3.1 and 3.2 for the corresponding methodology).

Metric	Value
Original records loaded	285
Removed records (input/label length mismatch)	5
Final records used for modelling	280
Training set size (70%)	196 recipes
Validation set size (30%)	84 recipes

The following assumptions were made while completing this assignment, in the absence of explicit instructions covering every edge case:

- Tokens in input and their corresponding labels in pos are aligned strictly by position after a simple whitespace split; no sub-word tokenisation (e.g. splitting “2-1/2” into separate tokens) is performed, since the label sequence is supplied at the whitespace-token level.
- Rows where the number of input tokens does not equal the number of pos labels are treated as corrupted / misaligned records and are dropped, since there is no reliable way to re-align them without ambiguity.
- The three labels observed in the data (ingredient, quantity, unit) are treated as the complete label set — no additional implicit label (e.g. an ‘O’ / outside class) is assumed to exist.
- A 70:30 train–validation split with a fixed random_state (42) is used for reproducibility. No separate held-out test set was supplied with the assignment, so validation performance is used as the best available proxy for generalisation.
- spaCy’s en_core_web_sm model is used to derive linguistic features (POS tag, lemma, dependency relation, shape) even though it is trained on general English text rather than recipe text — it still provides useful lexical and morphological signal for the CRF.

- The instruction to "penalise the ingredient label" during class-weight computation is implemented by halving the inverse-frequency weight computed for the majority ingredient class, so that its influence is reduced relative to the naturally rarer quantity and unit classes, without removing it entirely.

3. Methodology & Approach

The overall approach followed a standard supervised NLP sequence-labelling pipeline, implemented end-to-end in the accompanying Jupyter notebook:

3.1 Data Ingestion & Cleaning

The JSON file was loaded into a pandas DataFrame using a custom load_json_dataframe function. Each record's input and pos strings were tokenised by whitespace splitting into input_tokens and pos_tokens columns, and their lengths (input_length, pos_length) were compared to validate alignment.

This check revealed 5 out of 285 records (indices 17, 27, 79, 164, 207) where the token count and label count did not match. These records were removed, leaving 280 clean, fully-aligned recipes for modelling. A unique_labels helper confirmed that only three distinct labels (ingredient, quantity, unit) exist in the cleaned data — there is no label noise (e.g. typos or unexpected categories) to handle.

3.2 Train / Validation Split

The 280 cleaned recipes were split 70:30 into training (196 recipes) and validation (84 recipes) sets using scikit-learn's train_test_split with a fixed random seed, so that results are reproducible.

3.3 Exploratory Data Analysis

The input_tokens and pos_tokens lists were flattened across all recipes in each split, and tokens were categorised into ingredient / unit / quantity buckets using a categorize_tokens function. A get_top_frequent_items function (backed by collections.Counter) was then used to identify the 10 most frequent ingredient tokens and the 10 most frequent unit tokens, which were visualised as horizontal bar charts (see Section 4).

3.4 Feature Engineering for the CRF

Because a CRF predicts a label for every token conditioned on the whole sequence, each token was converted into a rich feature dictionary via a word2features function, combining:

- Core lexical/grammatical features — lower-cased token and lemma, spaCy POS tag, detailed tag, dependency relation, token shape, and boolean flags for stop-word, digit, punctuation, hyphenation, and slash presence.
- Quantity/unit detection features — boolean flags (is_quantity, is_unit, is_numeric, is_fraction, is_decimal) driven by curated unit_keywords and quantity_keywords sets plus a regex quantity_pattern that matches whole numbers, decimals, simple fractions ("1/2") and mixed fractions ("1-1/2").
- Contextual features — the previous and next token (and their quantity/unit status), plus explicit Beginning-Of-Sequence (BOS) and End-Of-Sequence (EOS) markers.

A sent2features function applied word2features across every token in a recipe, producing a per-recipe list of feature dictionaries, which were assembled into X_train_features / X_val_features.

3.5 Handling Class Imbalance

The training label distribution was heavily imbalanced: 5,323 ingredient tokens vs. 980 quantity tokens and 811 unit tokens. An inverse-frequency weight_dict was computed as total_samples ÷ label_count per label, and the ingredient weight was then halved to prevent it from being penalised too aggressively relative to the rarer classes. This weight was injected into every token's feature dictionary as an additional class_weight feature via extract_features_with_class_weights, giving the CRF an explicit signal about label rarity during training.

3.6 Model Training

A Conditional Random Field was trained using the sklearn_crfsuite.CRF wrapper around CRFsuite, with the following hyperparameters as specified in the assignment:

Hyperparameter	Value	Purpose
algorithm	lbfgs	Quasi-Newton optimisation for training
c1	0.5	L1 regularisation — encourages sparsity in feature weights
c2	1.0	L2 regularisation — penalises large weights, reduces overfitting
max_iterations	100	Cap on optimisation iterations
all_possible_transitions	True	Considers all label-to-label transitions for robustness

3.7 Evaluation & Error Analysis

The trained CRF was evaluated on both the training and validation sets using a flat classification report (precision, recall, F1 per label) and a confusion matrix. Misclassified validation tokens were then collected into an error DataFrame, capturing the token, its previous/next neighbours, its true and predicted labels, its class weight, and the full recipe context, to understand exactly where and why the model made mistakes.

4. Exploratory Data Analysis — Visualisations & Insights

4.1 Training Dataset

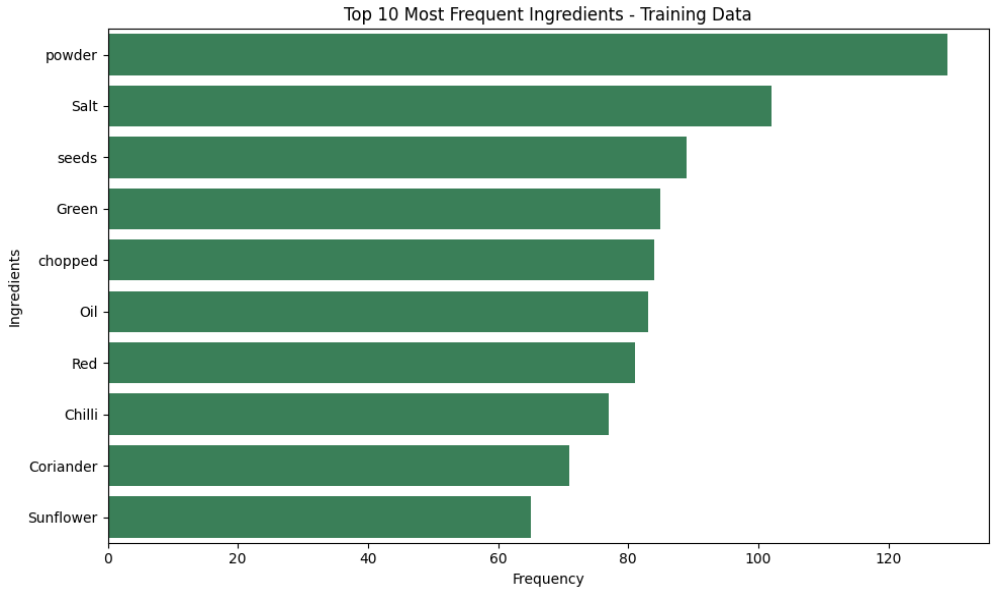

Figure 1: Top 10 most frequent ingredient tokens — Training data

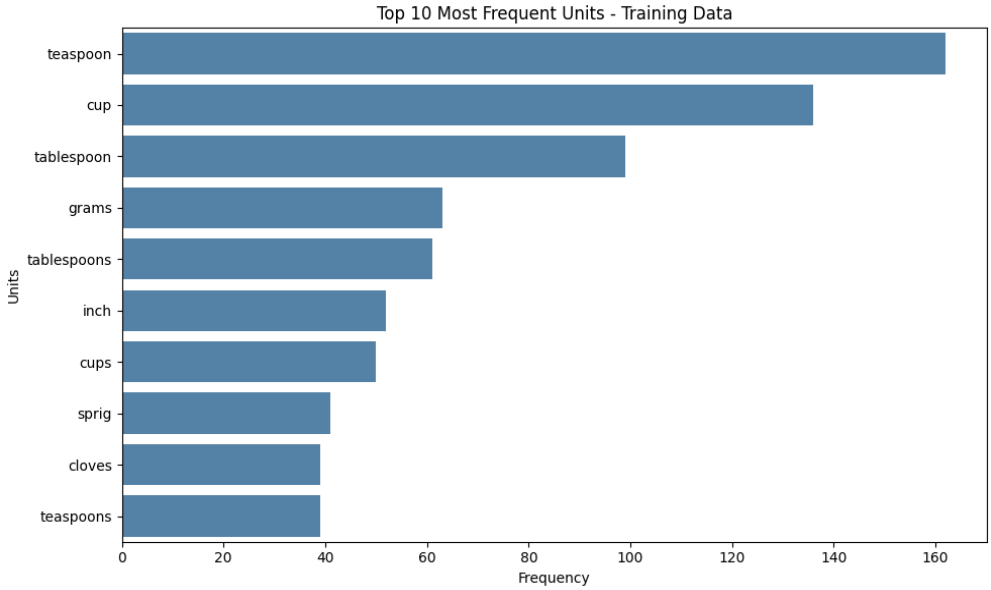

Figure 2: Top 10 most frequent unit tokens — Training data

The most frequent "ingredient" tokens are actually common descriptive words that appear inside longer ingredient phrases — powder, Salt, seeds, Green, chopped, Oil, Red, Chilli, Coriander, and Sunflower — reflecting that the recipes are predominantly Indian dishes that reference spice powders (turmeric / chilli / coriander powder), salt, chopped vegetables, and cooking oils very frequently. The most frequent units — teaspoon, cup, tablespoon, grams, tablespoons, inch, cups, sprig, cloves, teaspoons — are standard Indian-kitchen measurement units, with volumetric spoon/cup measures dominating over weight-based ones. Singular and plural forms of the same unit ("cup" / "cups") appear as separate top items because no normalisation was applied at the raw-token level; the lemma feature engineered for the CRF helps the model generalise across such variants.

4.2 Validation Dataset

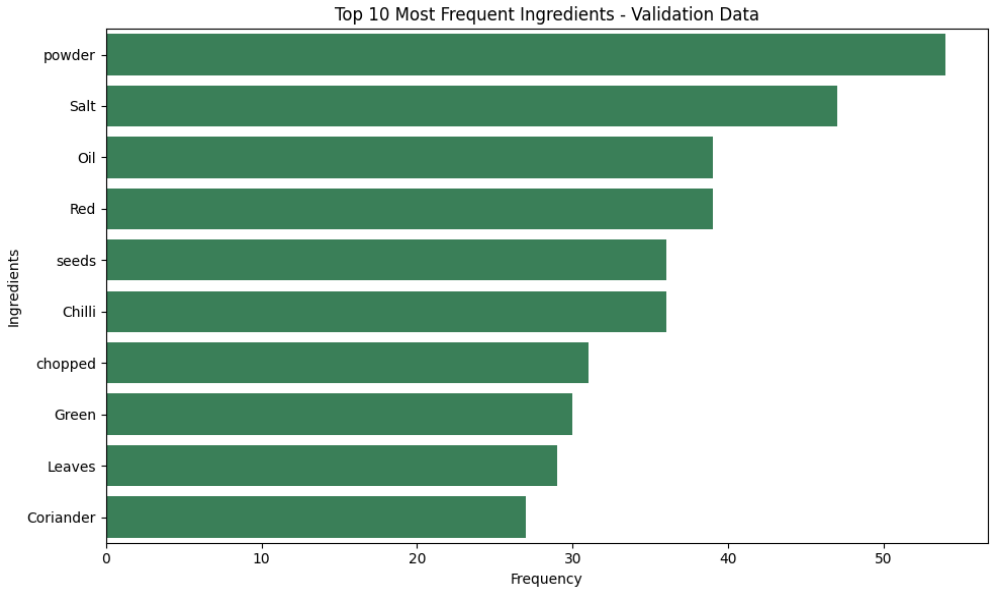

Figure 3: Top 10 most frequent ingredient tokens — Validation data

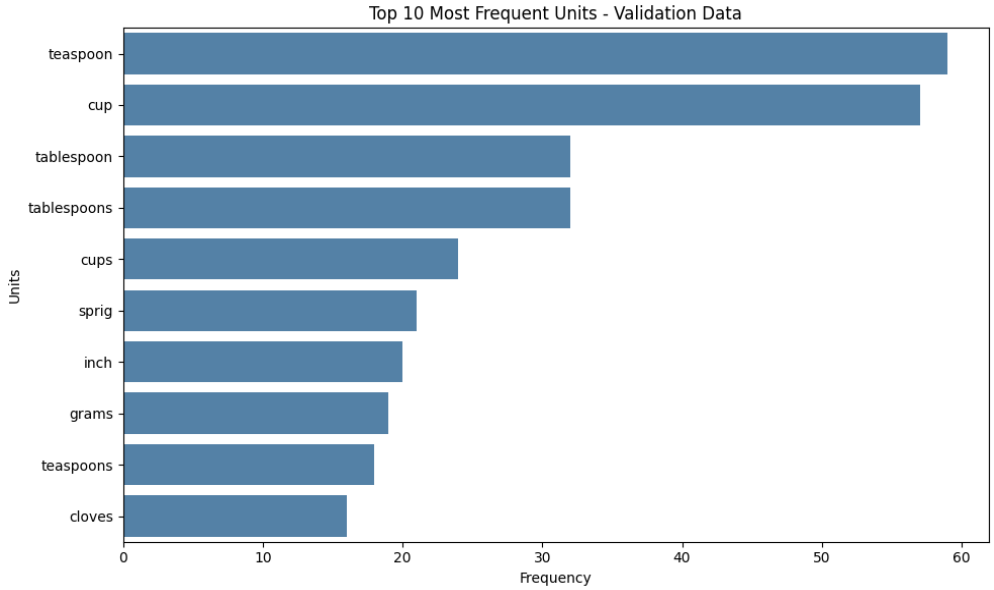

Figure 4: Top 10 most frequent unit tokens — Validation data

The validation set mirrors the training set's class distribution and vocabulary closely, which confirms that the random 70:30 split produced a representative validation sample rather than a skewed one — a good sign for the reliability of the validation metrics reported in Section 5.

5. Model Results

5.1 Training Set Performance

Label	Precision	Recall	F1-score	Support
quantity	1.000	0.996	0.998	980
unit	0.995	1.000	0.998	811
ingredient	1.000	1.000	1.000	5323
Overall accuracy			0.999	7114

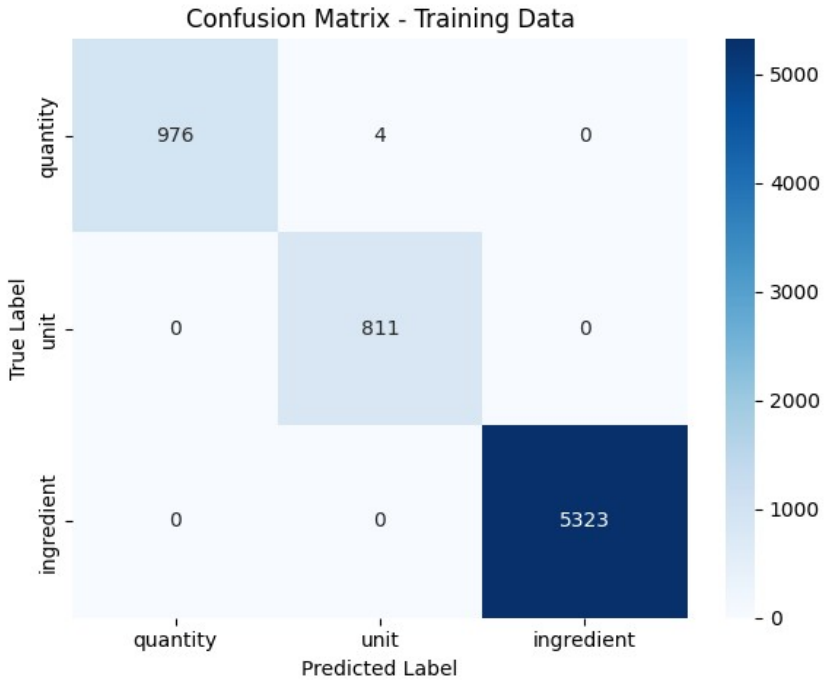

Figure 5: Confusion matrix — Training data

5.2 Validation Set Performance

Label	Precision	Recall	F1-score	Support
quantity	0.988	0.995	0.992	411
unit	0.994	0.986	0.990	358
ingredient	1.000	1.000	1.000	2107
Overall accuracy			0.998	2876

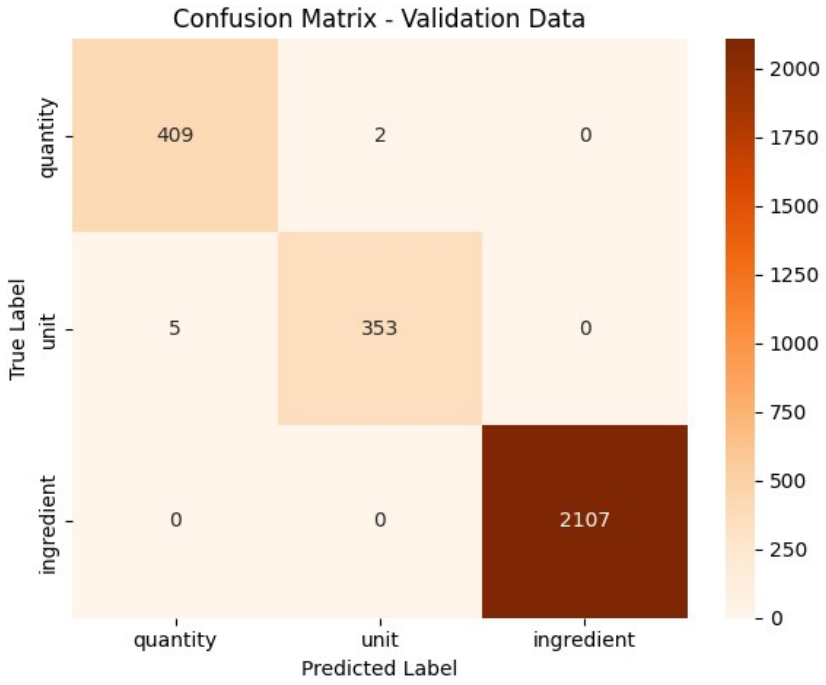

Figure 6: Confusion matrix — Validation data

The model generalises very well from training to validation, with validation accuracy of 99.76% (only 7 misclassified tokens out of 2,876) and F1-scores at or above 0.99 for every label. The ingredient class is classified with perfect precision and recall on both splits, while the rarer quantity and unit classes — despite being the minority classes — also achieve F1-scores above 0.99, indicating that the inverse-frequency class weighting strategy (Section 3.5) was effective.

6. Error Analysis

All 7 validation errors occur exclusively between the quantity and unit classes — the model never confuses an ingredient token with either a quantity or a unit, which is expected given that ingredient tokens (proper nouns / food names) are lexically very distinct from numeric or measurement tokens.

True label	Errors	Class weight	Total true count	Per-label accuracy
quantity	2	7.259	411	99.51%
unit	5	8.772	358	98.60%

Inspecting the specific misclassified tokens (“to”, “into”, “whole”, “and”, “a”, “pinch”, “cloves”) shows that confusion tends to occur with tokens that are inherently ambiguous outside their surrounding context — connector words that occasionally sit inside a quantity range phrase (e.g. “10 to 12”), and generic descriptors (“whole”, “pinch”, “cloves”) that can function as either a quantity/unit modifier or a plain ingredient descriptor depending on the surrounding words. This suggests that further accuracy gains would most likely come from richer contextual (wider window / n-gram) features around these specific ambiguous words, or a small amount of manual review of highly ambiguous edge cases, rather than from a fundamentally different model architecture.

7. Key Insights & Conclusion

- A class-weighted CRF with linguistically-informed features (spaCy POS/lemma/dependency/shape, regex-based quantity detection, keyword-based unit detection, and previous/next-token context) can achieve near-

perfect entity recognition (≈99.8% validation accuracy) on recipe ingredient text, even with a relatively small training set of under 200 recipes.

- Explicit domain-knowledge features (unit_keywords, quantity_keywords, quantity_pattern) contribute meaningfully alongside generic NLP features — encoding known measurement vocabulary directly into the feature space appears to be at least as valuable as relying purely on a general-purpose language model's POS tags.
- Class imbalance (ingredient tokens outnumber quantity and unit tokens roughly 5:1 and 6:1 respectively) did not prevent strong minority-class performance once inverse-frequency weighting was incorporated into the feature space.
- The small number of residual errors is concentrated at a specific, interpretable boundary (quantity vs. unit, around ambiguous connector/descriptor words), giving a clear, actionable direction for any future refinement of the model.

In summary, this assignment demonstrates a complete, reproducible pipeline — from raw JSON ingestion and data-quality validation, through EDA, feature engineering, class-weighted CRF training, and quantitative + qualitative evaluation — that is well-suited to power a structured recipe/ingredient database for applications such as dietary tracking, grocery-list generation, or recipe recommendation systems.

8. Limitations

While the class-weighted CRF achieves strong validation performance, several limitations should be kept in mind when interpreting or extending this work:

- Small, single-cuisine dataset — only 280 clean recipes (196 train / 84 validation), predominantly Indian dishes, so performance on cuisines with different ingredient/unit vocabulary (e.g. metric-only European recipes, or recipes written in other languages) is untested.
- No held-out test set — the assignment did not supply a separate test split, so the validation set doubles as the only proxy for generalisation; reported metrics may be mildly optimistic relative to unseen, truly out-of-sample data.
- spaCy features from a general-purpose model — en_core_web_sm was trained on general English text, not recipe or culinary text, so its POS/lemma/dependency features are an approximation rather than a domain-specific fit.
- Hand-curated keyword lists — unit_keywords and quantity_keywords were manually compiled and are not exhaustive; unusual or regional units/quantities outside these lists rely entirely on the model's contextual features.
- Only three entity labels — the model assumes every token is exactly one of ingredient, quantity, or unit; it has no 'other'/outside class, so it cannot flag tokens (e.g. connector words) that do not cleanly belong to any of the three categories.
- Class-weight penalty was manually tuned — halving the ingredient class weight was a heuristic choice rather than one selected via systematic hyperparameter search, so it may not be optimal for a different dataset or label distribution.

9. Reproducibility & Environment

9.1 Software / Library Versions

The pipeline was developed and run in a Google Colab notebook environment. Exact package versions are printed by running

!pip list (or *pip freeze*) inside the notebook, which is recommended before final submission so the exact resolved versions are captured alongside the results. The key libraries used are:

Library	Version used
Python	3.x (notebook run on Google Colab's default runtime)
pandas	as installed by the notebook's Colab runtime
scikit-learn	as installed by the notebook's Colab runtime
sklearn-crfsuite	0.5.0 (pinned in the notebook)
spaCy	with the en_core_web_sm English model
matplotlib / seaborn	as installed by the notebook's Colab runtime
joblib	as installed by the notebook's Colab runtime

9.2 Reproducibility Note

The full notebook, dataset, and this report are available in the project's code repository: https://github.com/bulah16/C5-Project

To reproduce the results in this report:

- Run the accompanying notebook (Identifying_Key_Entities_in_Recipe_Data_MBulah.ipynb) top to bottom in a fresh runtime, with ingredient_and_quantity.json placed in the same working directory.
- The train/validation split uses a fixed random_state = 42 in train_test_split, so the same 70:30 split (196 / 84 recipes) is reproduced on every run.
- The CRF itself (sklearn_crfsuite.CRF with algorithm='lbfgs') is a deterministic optimiser given the same features and labels, so model metrics should match to within minor floating-point differences across runs and machines.
- The only external, non-deterministic dependency is the spaCy en_core_web_sm model version; pinning it (e.g. via requirements.txt) is recommended if this pipeline is reused in a different environment.

10. References

Lafferty, J., McCallum, A., & Pereira, F. C. N. (2001). Conditional random fields: Probabilistic models for segmenting and labeling sequence data. *Proceedings of the 18th International Conference on Machine Learning*, 282–289.

Okazaki, N. (2007). *CRFsuite: A fast implementation of Conditional Random Fields (CRFs)* [Computer software documentation]. http://www.chokkan.org/software/crfsuite/

Honnibal, M., & Montani, I. (2017). spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing [Computer software]. Explosion AI. https://spacy.io

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Korobov, M. (n.d.). *sklearn-crfsuite documentation* [Computer software documentation]. Read the Docs. https://sklearn-crfsuite.readthedocs.io/